

SGLang使用

RTX 4090 推荐配置 (Qwen3-8B)



RTX 4090 (24GB) 跑 Qwen3-8B 的推荐配置

```
python -m sglang.launch_server \  
  --model-path /root/autodl-tmp/models/qwen/Qwen3-8B \  
  --port 8001 \  
  --host 0.0.0.0 \  
  --mem-fraction-static 0.85 \      # 显存利用率 85% (4090 留一些给系统)  
  --max-running-requests 16 \      # 最大并发请求数  
  --context-length 4096 \          # 上下文长度 (越短越省显存)  
  --chunked-prefill-size 2048 \    # 分块预填充 (降低 TTFT)  
  --enable-metrics                 # 开启监控指标
```

```
3-多轮对话.py ×
15
16 # 模拟多轮对话
17 user_inputs = [
18     "我的订单 2024001 到哪了? ",
19     "那这个订单可以退款吗? ",          # 第二轮: 复用第一轮 的 KV Cache
20     "退款需要多久到账? ",              # 第三轮: 复用前两轮 的 KV Cache
21 ]
22
23 for user_input in user_inputs:
24     messages.append({"role": "user", "content": user_input})
25
26     response = client.chat.completions.create(
27         model="Qwen3-0.6B",
28         messages=messages,
29         max_tokens=300,
30         temperature=0.7
31     )
32
33     reply = response.choices[0].message.content
34     messages.append({"role": "assistant", "content": reply})
35
36     print(f"用户: {user_input}")
37     print(f"客服: {reply}")
38     print("---")
39
```

sglang的中model跟 OpenAI client 中的model可以不一样吗？

```
response = client.chat.completions.create(
    model="Qwen3-0.6B",
    messages=messages,
    max_tokens=300,
    temperature=0.7
)
```

这是一个非常敏锐的观察！

在标准的使用规范下，**OpenAI Client** 中的 **model** 参数必须与 **SGLang** 后端实际加载的模型名称(或别名)保持一致。如果两者不一样，通常会导致 API 请求报错。

具体原因和底层机制如下：

1. **API 路由与校验机制**: SGLang 提供的是高度兼容 OpenAI 的 API 服务。当你执行 `client.chat.completions.create(model="Qwen3-0.6B")` 时，Client 只是将其作为一个 JSON 字段通过 HTTP 请求发给后端。SGLang 服务端接收到请求后，会校验这个请求要调用的模型是否存在于当前 GPU 显存挂载的实例库中。如果匹配不上，服务端会直接拦截并返回类似 404 Model not found 的 HTTP 错误响应。
2. **解释课件中的不一致**: 你上传的 image_52e020.jpg 截图中，启动命令加载的是 Qwen3-8B 模型；而附件 3-多轮对话.py 代码中请求的却是 Qwen3-0.6B 模型。这其实是因为它们属于课程的不同演示环节：Python 脚本对应的是前期为了快速调通流程而启动的 0.6B 小模型实例，而截图则是后续针对 RTX 4090 实战给出的 8B 大模型高阶配置建议。如果你用截图的命令启动了后端，再去跑这个脚本，必定会触发模型未找到的报错。
3. **如何动态获取准确的 Model ID**: 在企业级开发中，由于模型路径可能很长或包含哈希值，如果你不确定 SGLang 服务端究竟把哪个字符串注册为了 model 的名字，可以通过标准的查询接口获取：

`curl http://localhost:8001/v1/models` 系统会返回一个 JSON 列表，其中 `data[0].id` 的值就是你在 OpenAI Client 中需要精确填写的 model 字符串。

因此，如果你按截图部署了 Qwen3-8B，记得顺手把 Python 脚本里的 model 参数也改为对应的 "Qwen3-8B" 才能顺利跑通。